

AwIR - ćwiczenie 3

Temat:

Zapis danych formularza do bazy - wykorzystanie JPA

Cel ćwiczenia:

Rozbudowa aplikacji z ćwiczenia 2 o **trwałe przechowywanie danych** przy użyciu **Spring Data JPA** oraz dodanie funkcjonalności **zapisu, wyświetlania, edycji i usuwania użytkowników** w bazie danych.

Niezbędne narzędzia:

1. IDE: Spring Tool Suite 4 / IntelliJ IDEA / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Baza: wbudowana **H2 (in-memory)** lub dowolna lokalna

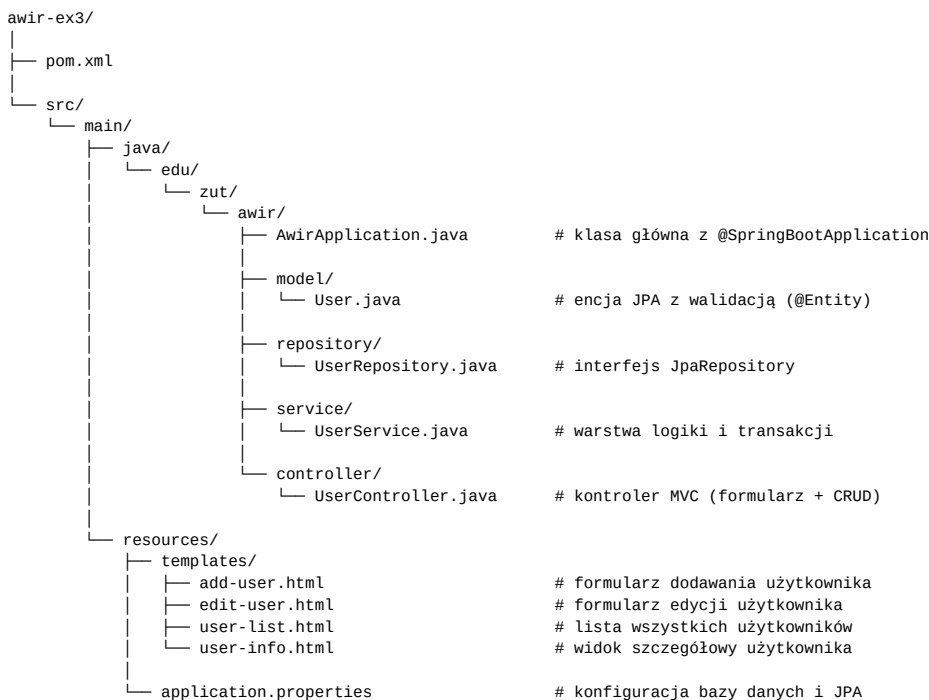
Projekt z ćwiczenia 2 zawierał:

- klasę modelową User,
- kontroler UserController,
- widoki add-user.html i user-info.html.

W tym ćwiczeniu:

- rozbuduj model User o adnotacje **JPA**,
- dodaj **repozytorium, serwis** i nowe metody w kontrolerze,
- skonfiguruj połączenie z bazą danych w application.properties.

Nowa struktura projektu



1. **Rozszerzenie zależności w pliku pom.xml**
spring-boot-starter-data-jpa

```
sterownik bazy danych, np.
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

2. Konfiguracja bazy danych (dataSource) w pliku application.properties

```
(skonfigurować zgodnie z używaną bazą danych)
spring.datasource.url=jdbc:h2:mem:db;DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.datasource.password=sa
spring.h2.console.enabled=true
spring.h2.console.path=/h2
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

3. Rozbudowa klasy domenowej

```
package edu.zut.awir.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import lombok.*;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be valid")
    private String email;
}
```

4. Repozytorium

Dodać interfejs UserRepository (edu.zut.awir.repository)

```
public interface UserRepository extends JpaRepository<User,
Long> {
    User findUserByName(String username);
}
```

5. Warstwa serwisu

Dodaj klasę serwisową w pakiecie edu.zut.awir.service.

```
package edu.zut.awir.service;

import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import edu.zut.awir.model.User;
import edu.zut.awir.repository.UserRepository;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository repository;

    @Transactional
    public User save(User user) {
        return repository.save(user);
    }

    @Transactional(readOnly = true)
    public List<User> findAll() {
        return repository.findAll();
    }

    @Transactional(readOnly = true)
    public User findById(Long id) {
        return repository.findById(id).orElse(null);
    }

    @Transactional
    public void delete(Long id) {
        repository.deleteById(id);
    }
}
```

6. Rozbudowa kontrolera

```
package edu.zut.awir.controller;

import edu.zut.awir.model.User;
import edu.zut.awir.service.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
```

```

@RequiredArgsConstructor
@RequestMapping("/users")
public class UserController {

    private final UserService service;

    @GetMapping("/new")
    public String showForm(Model model) {
        model.addAttribute("user", new User());
        return "add-user";
    }

    @PostMapping
    public String processForm(@Valid @ModelAttribute("user")
User user,
                                BindingResult result,
                                Model model) {
        if (result.hasErrors()) {
            return "add-user";
        }
        service.save(user);
        return "redirect:/users/list";
    }

    @GetMapping("/list")
    public String list(Model model) {
        model.addAttribute("users", service.findAll());
        return "user-list";
    }

    @GetMapping("/{id}/edit")
    public String editForm(@PathVariable Long id, Model model) {
        model.addAttribute("user", service.findById(id));
        return "edit-user";
    }

    @PostMapping("/{id}/edit")
    public String update(@PathVariable Long id,
                                @Valid @ModelAttribute("user") User
user,
                                BindingResult result) {
        if (result.hasErrors()) {
            return "edit-user";
        }
        user.setId(id);
        service.save(user);
        return "redirect:/users/list";
    }

    @PostMapping("/{id}/delete")
    public String delete(@PathVariable Long id) {
        service.delete(id);
        return "redirect:/users/list";
    }
}

```

```
}  
}
```

7. Widoki

- add-user.html – usuń pole id (bo generuje się automatycznie).
- Dodaj nowe pliki:

- user-list.html

```
<!DOCTYPE html>  
<html xmlns:th="http://www.thymeleaf.org">  
<body>  
<h2>Lista użytkowników</h2>  
<table border="1">  
  <tr><th>ID</th><th>Name</th><th>Email</th><th>Akcje</th></tr>  
  <tr th:each="u : ${users}">  
    <td th:text="${u.id}"></td>  
    <td th:text="${u.name}"></td>  
    <td th:text="${u.email}"></td>  
    <td>  
      <a th:href="@{/users/${u.id}/edit}">Edytuj</a>  
      <form th:action="@{/users/${u.id}/delete}"  
method="post" style="display:inline">  
        <button type="submit">Usuń</button>  
      </form>  
    </td>  
  </tr>  
</table>  
<a th:href="@{/users/new}">Dodaj nowego użytkownika</a>  
</body>  
</html>
```

- edit-user.html

```
<!DOCTYPE html>  
<html xmlns:th="http://www.thymeleaf.org">  
<body>  
<h2>Edytuj użytkownika</h2>  
<form th:action="@{/users/${user.id}/edit}" th:object="${user}" method="post">  
  <p>Imię: <input th:field="*{name}"/></p>  
  <p>Email: <input th:field="*{email}"/></p>  
  <button type="submit">Zapisz zmiany</button>  
</form>  
</body>  
</html>
```

8. Uruchomienie i test

Adresy:

- http://localhost:8080/users/new – formularz dodawania
- http://localhost:8080/users/list – lista użytkowników
- http://localhost:8080/h2 – konsola bazy danych